1

Double Flop Synchronizer : (As show in Fig 1)

① F1 samples the asynchronous input signal into the destination clock domain, probably a meta-stable value.

② With a full clk2 cycle to permit any meta-stability on the F1 output signal to decay.

③ Then the signal is sampled by the same clock into F2, which we expect to be a stable and valid signal.

Fig 1 Double Flop Synchronizer

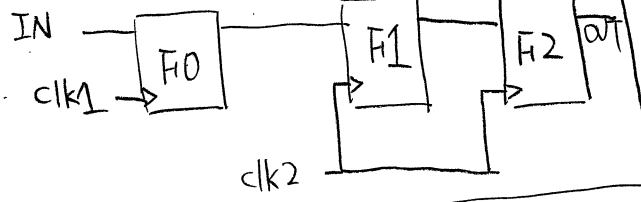
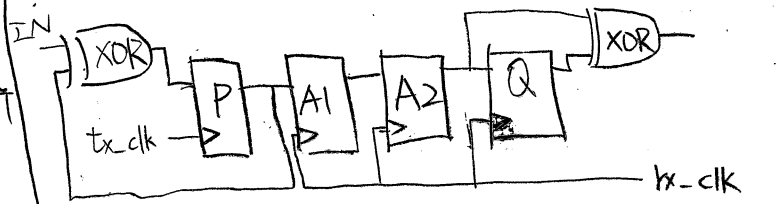


Fig 2 XOR double Flop Synchronizer



XOR double Flop Synchronizer : (Fig 2)

The additional circuit, XOR gate, is used to ensure the input, clock domain tx_clk, "Pulse" can be transmitted to the output, clock domain rx_clk,

---

When the frequency of two clock domain are different, the clock skew between clocks varies with time. The skew leads to problem that sometimes setup/hold check violated, and sometimes not. This will cause the receiving flip-flop in the destination domain enter metastable state.

Thus, we need to use synchronizer to sample the signal from source domain to destination domain.

2.

For synthesis:

We have to release the timing check and inform the synthesizer, we can either directly write in syn.tcl or write in syn.sdc and read that file in syn.tcl. Both setting multicycle path and setting false path make the timing check are OK, but setting multicycle is recommended. Setting multicycle path make the timing check looser in certain pathes, wheras setting false path just does not check the pathes in certain path. Both methods make the timing check in synthesis level.

For gate level simulation:

We have to generate the sdf file that the first flip-flop in received clock domain have no setup and hold timing check. As mentioned in Question 1, this flip-flop will easily face the problem of setup/hold timing violation and enter the metastable state. It is OK since the 2nd flip-flop will not. However, the simulator does not know and if setup/hold timing violation occur, the output will be unknow. Thus the setup/hold timing check of the flip-flop in recieved clock domain should be set to. "0" in .sdf file

3. Name of the flow	Required Information.	Rank of Presision
VCD Flow (Value change dump)	1 Power Models 2. Netlist and Parasitics 3. Signal Activity (VCD)	1st
SAIF Flow (Switching Activity Interchange Format)	1 Power Models 2. Netlist and Parasitics 3 Signal Activity (SAIF)	2nd
Vector free Flow	1 Power Models 2. Netlist and Parasitics	3rd

VCD file : Value change dump file, VCD file contains value changes of signal, ie. at what times signals changes their values

SAIF file : Switching Activity Interchange Format file, SAIF contains toggle counts and time information like how much time a signal was in 1 state, 0 state, x state.
It contains cumulative information of vcd.

4.

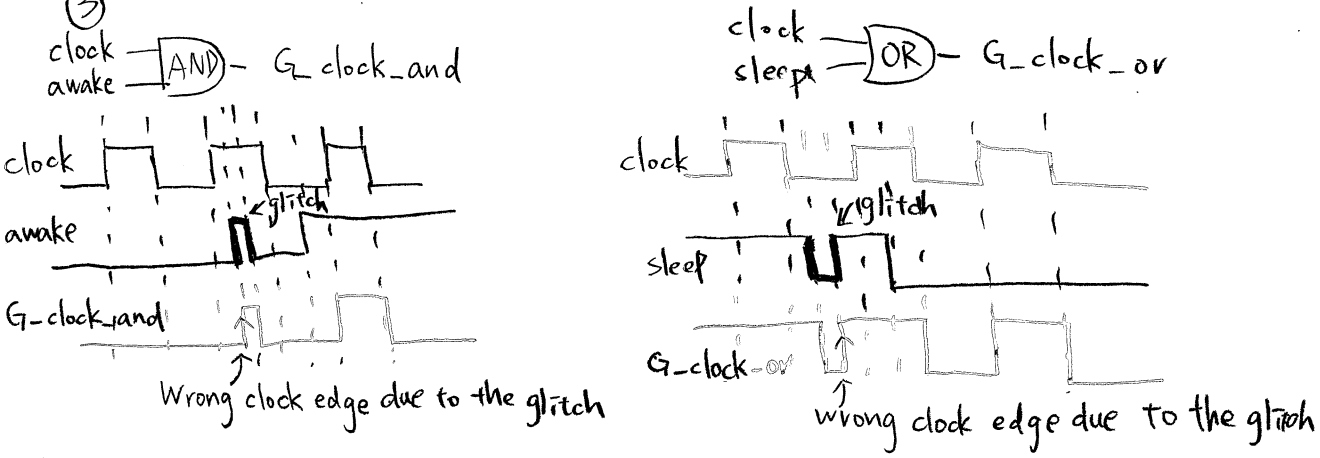
① OR-gate is better.

② OR-gate = The gated clock is tied high when register are turned off.
Thus the first latch circuit will not toggle while data is toggling.

AND-gate = The gated clock is tied low when register are turned off.
Thus, the first latch circuit will toggle while data is toggling.

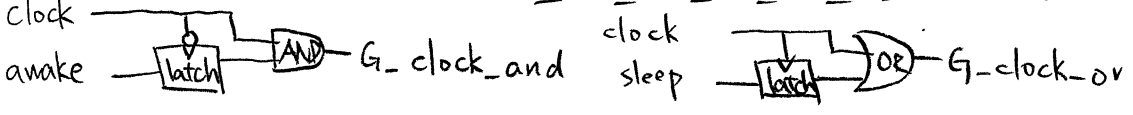
⇒ Therefore, AND-gate consumes more power than OR-gate.

③



As shown in above figures, for AND-based clock gating cells, the awake signal cannot rise when clock is 1. Since AND-base clock gating cells try to tie the output to "0", the awake signal passes when clock is "1" and when awake is rising the flip-flop will be triggered. As the glitch contains both rising and falling edge, the glitch is not allowed when clock is "1".

Similar to AND-based clock gating cells. OR-based clock gating cells, the output is tied to "1", thus the sleep signal cannot rise when clock is "0". Therefore the glitch is not allowed when clock is "0"



Solution:
Add latches can gate awake/sleep signals when clock is 1/0, which are the non-allowable changing regions, Thus, the trigger can be eliminated or occure at correct time.

5.

(1) typedef : allows users to create their own name for type defination that they code to make code clear.

(2) enum : Defined a set of named values which provides built-in assertion

6.

(1) pre_randomize() : Set/Correct constraints before randomization.

Post_randomize() : Make corrections after randomization.

Both pre_randomize() and post_randomize() are called when randomize() executes.

```
function void pre_randomize();
  "Code your code here"
endfunction.
```

```
function void post_randomize()
  "Code your code here"
endfunction
```

(2)

```
class random_val;
    bit [31:0] array [3] = {32'h1000_0000,
                            32'h0100_0000,
                            32'h0000_0001};
    bit [31:0] rand_val;
  randc bit [1:0] index;
  constraint limit
  {
    index inside {0, 1, 2};
  }
  function void post_randomize();
        rand_val=array [index];
  endfunction
  endclass
```

7.

```
Assertion 1 : assertion property (@(posedge clk)
                                  (A |-> ## [1 : $] (C && !B)))

else begin
    $fatal;
end

Assertion 2 : assertion property (@(posedge clk)
                                  ($fell(A) |-> ## [2 : 10] B[*10]))

else begin
    $fatal;
end

Assertion 3 : assertion property (@(posedge clk)
                                  (!A |-> $past(B, 2) == 1))

else begin
    $fatal;
end
```

8.

data : 5'b0_1000 ⇒ 5'b1_0010 ⇒ 5'b0_1010 ⇒ 5'b1_1011 ⇒ 5'b1_0011

bit 0 of data toggle : 1 out of 2

bit 1 of data toggle : 1 out of 2

bit 2 of data toggle : 0 out of 2

bit 3 of data toggle : 2 out of 2

bit 4 of data toggle : 2 out of 2

$$\frac{1+1+2+2}{2+2+2+2+2} = \frac{6}{10} = 60\%$$

9.

LIB Library : (.lib)

① Timing information
② Calculate cell delay according to different combinations of input slew rate and output loading capacitance
③ Using Signal Storm Library Characterizer

LEF Library : (.lef)

① Process and cell information
② Metal layer information; routing pitch, default direction.
③ Cell size, pin position, pin metal layer

RC Extraction : (.tch, captbl)

RC extration for further power analysis, simulation ... etc.

Celtic Library : (.cdb)

CdB model (noise model for each cell which will be use in SI optimization phase)

GDSII : (.gds2)

Planar geometric shapes and other information about the layout in hierarchical form.

Chip design netlist (.v)

Combined the netlist after synthesized and the netlist contains I/O, I/O power, and core power.

IO pad loction file : (.io)

Describe IO pad location

Timing Constraint files (.sdc)

Include timing / driving / loading constraint information

10.

① Signal Integrity Issue :

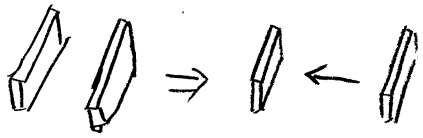
Include crosstalk, charge sharing, supply noise, leakag, propagated noise overshoot, and under shoot issues.

These issues can degrade the electrical signal to the point where errors occur and system or device fails.

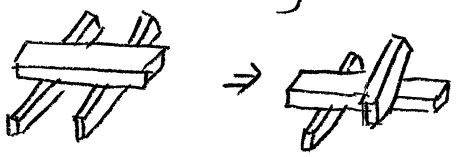
②

Increasing the wiring spacing, switching the same layer to another layer reducing parallel wires and re-ordering net can prevent SI issue

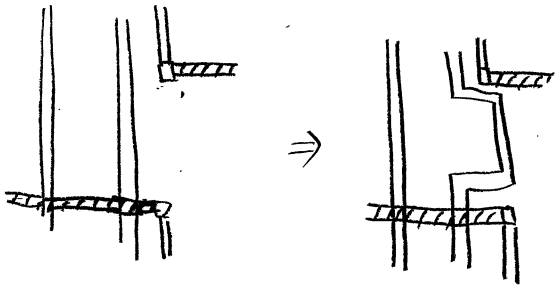
Wiring spacing



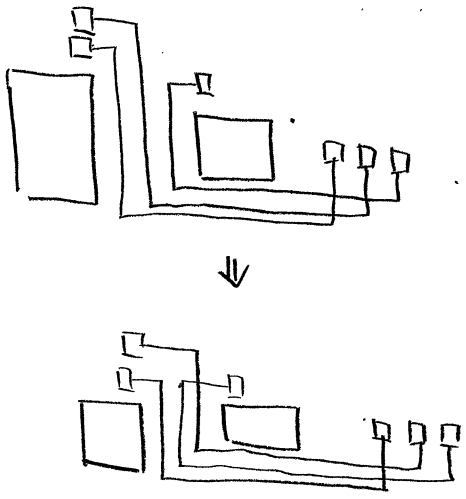
Layer switching



Parallel wires reducing



Re-ordering net



11.

① Serious

② IR drop is the problem of voltage drop of the power and ground due to high current flowing through the power-ground resistive network. Thus, as the proces more advanced, the minimum width of the wire becomes more narrow and the length of the routing becomes longer. Moreover the switch loss caused by high speed operation and leakage power are increase.

Therefore, it is become serious as the process improved

12.

a.

Power analysis : Analyze the power consumption of the design

Rail analysis : Analyze if the powerplan is enough to provide whole chip.

b.

Vector-based power analysis:

use VCD (waveform file), like .fsdb and .vcd

It require

① Simulation is possible at the full-chip level.

② Simulation provides sufficient functional coverage of design.

③ Vectors include these cause highest power consumption.

Rail analysis:

static-VDD.ptiavg : power/current file.

static_GND.ptiavg : power/current file.

c. Yes

It's depend on the waveform file (.vcd or .fsdb)

Thus, the diffence of coverage or input vector must provide the different result.